

Gradientes, autodiferenciación y optimización

Guía de estudio y comprensión

M08 · Matemáticas y Programación para IA

PROPÓSITO

Esta guía acompaña las 43 diapositivas del módulo y sigue exactamente su secuencia conceptual. Amplía la intuición, la notación, las ecuaciones, los ejemplos numéricos y las conexiones necesarias para comprender el material sin depender únicamente de lo que cabe en cada slide.

CÓMO USARLA

Revisa primero la diapositiva correspondiente y después usa esta página para reconstruir el razonamiento. Las secciones **Primero entiéndelo así**, **Qué debes comprender**, **Matemática esencial**, **Interpretación**, **Idea clave** y **Cuidado** están pensadas para convertir cada símbolo abstracto en una idea concreta.

Mapa del módulo

Parte	Slides	Núcleo conceptual
I	1–3	Pérdida, derivadas parciales y gradiente.
II	4–6	Aproximación local, dirección y descenso.
III	7–10	Regla de la cadena y backpropagation.
IV	11–13	DAG, micrograd y orden topológico.
V	14–19	Lotes, PyTorch, gradientes y actualización.
VI	20–23	Estabilidad de Gradient Descent.
VII	24–30	Mini-lotes, Momentum y Adam.
VIII	31–37	train/eval y registro del grafo.
IX	38–43	Ciclo de entrenamiento, evaluación y diagnóstico.

I · Pérdida, derivadas y gradiente

Slide 1 - Gradientes, autodiferenciación y optimización

PRIMERO ENTIÉNDELO ASÍ

Cuando un modelo se equivoca hay tres tareas distintas. El **gradiente** describe cómo cambiaría la pérdida si modificáramos ligeramente los parámetros actuales. **Backpropagation** organiza el cálculo de esa información desde la pérdida hasta cada parámetro. El **optimizador** usa los gradientes para decidir el cambio efectivo de los parámetros.

QUÉ DEBES COMPRENDER

La palabra *ligeramente* es importante: al hablar del gradiente estamos describiendo lo que ocurre alrededor de los valores actuales de los parámetros, no el comportamiento de la función en cualquier punto lejano.

Matemática esencial

$$\text{parámetros } \theta \longrightarrow \text{predicción} \longrightarrow L \xrightarrow{\text{backward}} \nabla_{\theta} L$$

Interpretación

θ reúne los parámetros entrenables; L es una pérdida escalar; $\nabla_{\theta} L$ reúne una derivada por cada parámetro. El gradiente todavía no es la actualización: el optimizador usa esa información junto con una regla de paso.

IDEA CLAVE

Separar las responsabilidades evita una confusión común: `backward()` calcula gradientes; por sí solo no modifica los parámetros.

CUIDADO CON ESTA CONFUSIÓN

Gradiente, backpropagation y optimizador están conectados, pero no son sinónimos.

I · Pérdida, derivadas y gradiente

Slide 2 - Una pérdida escalar organiza todo el entrenamiento

PRIMERO ENTIÉNDELO ASÍ

Antes de calcular derivadas necesitamos una cantidad que resuma qué tan mal está funcionando el modelo. Esa cantidad es la **pérdida**. En el ejemplo, los datos x e y permanecen fijos; los parámetros w y b son los que pueden cambiar durante el entrenamiento.

QUÉ DEBES COMPRENDER

Con una observación fija, el modelo produce una predicción y la pérdida convierte el error de esa predicción en un único número que queremos reducir.

Matemática esencial

$$\hat{y} = wx + b, \quad r = \hat{y} - y, \quad L = \frac{1}{2}r^2.$$

$$x = 2, y = 5, (w_0, b_0) = (1, 0) \Rightarrow \hat{y}_0 = 2, r_0 = -3, L_0 = 4.5.$$

Interpretación

x es la entrada, y el valor correcto, $\hat{y}$ la predicción, r el residuo y L la pérdida. Al cambiar w o b cambian $\hat{y}$, r y finalmente L .

Ejemplo breve

Si mantenemos $w = 1$ y cambiamos b de 0 a 1, la predicción pasa de 2 a 3, el residuo de -3 a -2 y la pérdida de 4.5 a 2.

IDEA CLAVE

Entrenar significa buscar parámetros que produzcan una pérdida menor sobre los datos usados para el entrenamiento.

CUIDADO CON ESTA CONFUSIÓN

El valor objetivo y no es la función objetivo L : y es el dato correcto; L es la cantidad que se minimiza.

I · Pérdida, derivadas y gradiente

Slide 3 - El signo anticipa la derivada antes del cálculo

PRIMERO ENTIÉNDELO ASÍ

En $(w, b) = (1, 0)$ la predicción es 2 y el objetivo es 5: el modelo está por debajo. Antes de derivar podemos probar qué ocurre si aumentamos un poco un parámetro mientras el otro permanece fijo.

QUÉ DEBES COMPRENDER

Una **derivada parcial** responde exactamente a esa idea: si cambio ligeramente un parámetro y mantengo los demás fijos, ¿cómo cambia la pérdida alrededor del punto actual?

Matemática esencial

$$w : 1 \rightarrow 1.1 \Rightarrow \hat{y} = 2.2, \quad b : 0 \rightarrow 0.1 \Rightarrow \hat{y} = 2.1.$$

$$\left. \frac{\partial L}{\partial w} \right|_{(1,0)} < 0, \quad \left. \frac{\partial L}{\partial b} \right|_{(1,0)} < 0.$$

$$\theta = \begin{bmatrix} w \\ b \end{bmatrix}, \quad \nabla_{\theta} L = \begin{bmatrix} \partial L / \partial w \\ \partial L / \partial b \end{bmatrix}.$$

Interpretación

Un valor negativo de $\partial L / \partial w$ significa que, alrededor del punto actual, aumentar un poco w hace disminuir L si lo demás queda fijo. El gradiente simplemente reúne todas esas derivadas parciales en un vector.

IDEA CLAVE

El signo de una derivada describe el sentido del cambio de la pérdida; no indica que el parámetro sea negativo.

CUIDADO CON ESTA CONFUSIÓN

La conclusión es local: un aumento pequeño puede reducir la pérdida, pero eso no garantiza que un aumento arbitrariamente grande siga haciéndolo.

II · Cambios pequeños y descenso

Slide 4 - El gradiente predice cambios locales de la pérdida

PRIMERO ENTÉNDELO ASÍ

Tenemos parámetros actuales θ y proponemos un cambio pequeño $\Delta\theta$. El gradiente permite estimar cuánto cambiaría la pérdida sin tener que evaluar inmediatamente todos los cambios posibles.

QUÉ DEBES COMPRENDER

Aquí **local** significa *alrededor de los valores actuales y para cambios pequeños*. La expresión es una aproximación de primer orden, no una descripción exacta de toda la función.

Matemática esencial

$$L(\theta + \Delta\theta) \approx L(\theta) + \nabla L(\theta)^T \Delta\theta.$$

$$\Delta L_{\text{lin}} = \nabla L(\theta)^T \Delta\theta = \frac{\partial L}{\partial w} \Delta w + \frac{\partial L}{\partial b} \Delta b.$$

Interpretación

θ es el punto actual; $\Delta\theta$ es el cambio propuesto; $\nabla L(\theta)$ se calcula en el punto actual. El producto interno resume el efecto conjunto de pequeños cambios en todos los parámetros.

Ejemplo breve

Si $\nabla L = (-6, -3)^T$ y solo cambiamos w en 0.01, entonces $\Delta L_{\text{lin}} = (-6)(0.01) = -0.06$. La pérdida se espera que disminuya aproximadamente 0.06.

IDEA CLAVE

El gradiente funciona como un modelo lineal de lo que ocurrirá con la pérdida cerca del punto actual.

CUIDADO CON ESTA CONFUSIÓN

La aproximación predice bien cambios pequeños; para cambios grandes la curvatura de la función puede hacer que el valor exacto se aparte mucho de la predicción lineal.

II · Cambios pequeños y descenso

Slide 5 - Descenso local máximo: contra el gradiente

PRIMERO ENTIÉNDELO ASÍ

Si queremos movernos una distancia pequeña y solo decidir *hacia dónde*, conviene separar dirección y tamaño del paso. Para eso usamos un vector d de longitud uno.

QUÉ DEBES COMPRENDER

$\|d\|_2 = 1$ significa que d está normalizado: su longitud euclídea es uno. Así comparamos únicamente hacia dónde apunta el movimiento, no qué tan grande es el paso.

Matemática esencial

$$D_d L(\theta) = \nabla L(\theta)^T d, \quad \|d\|_2 = 1.$$

$$\nabla L(\theta)^T d \geq -\|\nabla L(\theta)\|_2, \quad d^* = -\frac{\nabla L(\theta)}{\|\nabla L(\theta)\|_2}.$$

Interpretación

$D_d L$ es la derivada direccional: estima cómo cambia L si avanzamos una cantidad muy pequeña en la dirección d . Entre todas las direcciones de longitud uno, la opuesta al gradiente produce el cambio lineal más negativo.

Ejemplo breve

Si $d = (3/5, 4/5)^T$, entonces $\|d\|_2 = \sqrt{(3/5)^2 + (4/5)^2} = 1$. El subíndice 2 indica la norma euclídea; no significa elevar el vector al cuadrado.

IDEA CLAVE

$-\nabla L$ decide una dirección de descenso de primer orden; la tasa de aprendizaje decidirá después cuánto avanzar.

CUIDADO CON ESTA CONFUSIÓN

Si $\nabla L = 0$, la expresión normalizada no está definida. Además, *máximo descenso* aquí se refiere al modelo local de primer orden y a la norma euclídea.

II · Cambios pequeños y descenso

Slide 6 - La cuadrática contrasta dirección y cambio de pérdida

PRIMERO ENTIÉNDELO ASÍ

La pérdida cuadrática permite comprobar con números qué ocurre si avanzamos a favor o en contra del gradiente. Así distinguimos la predicción lineal del cambio real de una función curva.

QUÉ DEBES COMPRENDER

En el punto $(w, b) = (1, 0)$ del ejemplo, $\nabla L = (-6, -3)^T$. Un paso a favor del gradiente debería aumentar la pérdida; un paso en contra debería reducirla.

Matemática esencial

$$\begin{aligned} L(w, b) &= \frac{1}{2}(wx + b - y)^2, & L(1, 0) &= 4.5. \\ 0.05\nabla L &= (-0.30, -0.15), & \Delta L_{\text{lin}} &= +2.25. \\ -0.05\nabla L &= (0.30, 0.15), & \Delta L_{\text{lin}} &= -2.25. \end{aligned}$$

Interpretación

El signo de $\nabla L^T \Delta \theta$ predice si el cambio pequeño subirá o bajará la pérdida. En la cuadrática, el cambio exacto incluye además términos de segundo orden, por eso no coincide necesariamente con el valor lineal.

Ejemplo breve

Paso a favor: L pasa de 4.5 a 7.03125. Paso en contra: L pasa de 4.5 a 2.53125. El signo previsto es correcto, aunque el tamaño exacto del cambio no coincide con ± 2.25 .

IDEA CLAVE

La aproximación lineal es especialmente útil para anticipar el signo del cambio cerca del punto actual.

CUIDADO CON ESTA CONFUSIÓN

No interpretes ΔL_{lin} como el cambio exacto cuando la función tiene curvatura.

III · Backpropagation

Slide 7 - Backpropagation empieza por un forward descompuesto

PRIMERO ENTIÉNDELO ASÍ

Una expresión compleja puede verse como una secuencia de operaciones simples. Esa descomposición forma un grafo computacional y permite conocer la derivada local de cada operación.

QUÉ DEBES COMPRENDER

Una **derivada local** describe una sola operación respecto de su entrada inmediata. Todavía no es la derivada total de la pérdida respecto de un parámetro lejano.

Matemática esencial

$$u = wx, \quad \hat{y} = u + b, \quad r = \hat{y} - y, \quad q = r^2, \quad L = \frac{1}{2}q.$$

$$\frac{\partial u}{\partial w} = x, \quad \frac{\partial \hat{y}}{\partial u} = 1, \quad \frac{\partial r}{\partial \hat{y}} = 1, \quad \frac{\partial q}{\partial r} = 2r, \quad \frac{\partial L}{\partial q} = \frac{1}{2}.$$

Interpretación

Con $x = 2, y = 5, w = 1, b = 0$ el forward produce $u = 2, \hat{y} = 2, r = -3, q = 9$ y $L = 4.5$. Cada flecha del grafo tiene una regla de derivación simple.

IDEA CLAVE

Backpropagation funciona porque una función grande se puede descomponer en operaciones pequeñas y luego conectar sus derivadas mediante la regla de la cadena.

CUIDADO CON ESTA CONFUSIÓN

$\partial u / \partial w = 2$ no es todavía $\partial L / \partial w$. Para llegar a la pérdida hay que incorporar todas las operaciones posteriores.

III · Backpropagation

Slide 8 - La sensibilidad nace en 1 y viaja hacia atrás

PRIMERO ENTIÉNDELO ASÍ

El recorrido inverso comienza en la salida escalar. La semilla es $\partial L / \partial L = 1$ porque si L cambia una unidad, respecto de sí misma cambia una unidad.

QUÉ DEBES COMPRENDER

En este contexto, **sensibilidad** significa: si este nodo intermedio cambia una cantidad muy pequeña, ¿cuánto cambia finalmente la pérdida? El recorrido hacia atrás calcula precisamente esas cantidades.

Matemática esencial

$$\frac{\partial L}{\partial L} = 1, \quad \frac{\partial L}{\partial q} = \frac{1}{2}, \quad \frac{\partial L}{\partial r} = \frac{1}{2}(2r) = r = -3.$$

$$\frac{\partial L}{\partial \hat{y}} = -3, \quad \frac{\partial L}{\partial u} = -3, \quad \frac{\partial L}{\partial w} = \frac{\partial L}{\partial u} \frac{\partial u}{\partial w} = (-3)(2) = -6, \quad \frac{\partial L}{\partial b} = -3.$$

Interpretación

Cada paso multiplica la sensibilidad recibida por la derivada local de la operación. La regla de la cadena permite conectar el efecto de un nodo intermedio con el efecto final sobre L .

Ejemplo breve

Como $L = \frac{1}{2}q$, $\partial L / \partial q = 1/2$: un cambio pequeño de q produce un cambio de la mitad en L .

IDEA CLAVE

La semilla escalar es 1; a partir de ella se encadenan derivadas locales hasta llegar a los parámetros.

CUIDADO CON ESTA CONFUSIÓN

La sensibilidad de un nodo y la derivada local de una arista son conceptos distintos: una mide el efecto total sobre L ; la otra describe una operación inmediata.

III · Backpropagation

Slide 9 - Dos rutas aportan; el gradiente se suma

PRIMERO ENTIÉNDELO ASÍ

Una misma variable puede influir en la salida por más de una ruta. El efecto total debe incluir todas las rutas, no escoger solo una.

QUÉ DEBES COMPRENDER

Cuando un nodo compartido recibe contribuciones desde varias ramas del grafo, backpropagation las **acumula**. Acumular aquí significa sumar efectos que pertenecen al mismo forward y al mismo backward.

Matemática esencial

$$z = a^2 + a, \quad \frac{dz}{da} = \frac{d(a^2)}{da} + \frac{da}{da} = 2a + 1.$$

Interpretación

La rama a^2 aporta $2a$ y la rama directa aporta 1. Con $a = 2$, los aportes son 4 y 1, por lo que la derivada total es 5.

IDEA CLAVE

En un grafo con ramas, la derivada total de un nodo es la suma de las contribuciones de todas las rutas que dependen de él.

CUIDADO CON ESTA CONFUSIÓN

Esta suma de ramas no es lo mismo que acumular gradientes entre llamadas distintas a `backward()`; esa acumulación aparece después en PyTorch.

III · Backpropagation

Slide 10 - Backpropagation parte de una salida escalar

PRIMERO ENTÉNDELO ASÍ

Si la salida final es un escalar, el recorrido inverso tiene una semilla natural: la derivada del escalar respecto de sí mismo vale 1. Si la salida es un vector, primero hay que especificar qué cantidad escalar construida a partir de sus componentes se desea diferenciar, o proporcionar directamente la sensibilidad inicial correspondiente.

QUÉ DEBES COMPRENDER

Para una salida vectorial no existe un único gradiente respecto de los parámetros que represente simultáneamente todas sus componentes. El objeto completo es un Jacobiano. Reverse-mode evita construirlo cuando solo necesitamos una combinación concreta de sus filas.

Matemática esencial

$$y(\theta) = \begin{bmatrix} y_1(\theta) \\ y_2(\theta) \end{bmatrix}, \quad s = y_1 + 2y_2, \quad \frac{\partial s}{\partial y} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

$$\nabla_{\theta} s = \left(\frac{\partial s}{\partial y} \right)^T J_y(\theta).$$

Interpretación

$J_y(\theta)$ describe cómo cambia cada componente de y respecto de cada parámetro. El vector $(1, 2)^T$ expresa cómo el escalar s depende de y_1 y y_2 . El resultado $\nabla_{\theta} s$ responde cómo cambia ese escalar final al modificar cada parámetro.

Ejemplo breve

En clasificación multiclase, el modelo produce un vector de logits. La función de pérdida, por ejemplo cross-entropy, combina ese vector con la etiqueta correcta y produce un escalar L . Entonces `loss.backward()` puede comenzar con $\partial L / \partial L = 1$.

IDEA CLAVE

Reverse-mode calcula productos vector-Jacobiano sin necesidad de materializar el Jacobiano completo.

CUIDADO CON ESTA CONFUSIÓN

No confundas la salida vectorial del modelo con la etiqueta de clase. El vector contiene logits o probabilidades; la etiqueta correcta es un dato separado.

IV · DAG y autodiferenciación

Slide 11 - micrograd expone el mismo DAG escalar**PRIMERO ENTIÉNDELO ASÍ**

micrograd permite observar con pocos objetos el mismo mecanismo conceptual que usan motores de autodiferenciación más grandes: valores, dependencias, operaciones locales y gradientes acumulados.

QUÉ DEBES COMPRENDER

Un **DAG** es un grafo dirigido acíclico: las operaciones tienen una dirección de dependencia y no forman ciclos en el forward. Cada nodo sabe de qué nodos anteriores depende.

Matemática esencial

$$w, x \rightarrow u = wx \rightarrow \hat{y} = u + b \rightarrow r = \hat{y} - y \rightarrow q = r^2 \rightarrow L = \frac{1}{2}q.$$

Interpretación

Un objeto tipo `Value` contiene el valor calculado en el forward, referencias a sus padres, la operación que lo creó y un campo donde se acumula el gradiente durante el backward.

IDEA CLAVE

micrograd no cambia la matemática: hace visible el grafo y el recorrido que normalmente quedan ocultos dentro de una biblioteca.

CUIDADO CON ESTA CONFUSIÓN

Su valor principal aquí es conceptual. No debe interpretarse como una API de producción equivalente a PyTorch.

IV · DAG y autodiferenciación

Slide 12 - La traza manual fija lo que micrograd debe reproducir

PRIMERO ENTIÉNDELO ASÍ

Antes de confiar en un motor automático conviene conocer los valores que debería producir el ejemplo. La traza manual sirve como referencia para comprobar tanto el forward como el backward.

QUÉ DEBES COMPRENDER

La evidencia esperada incluye dos tipos de datos: el valor de cada nodo en el forward y la derivada total de la pérdida respecto de ese nodo en el backward.

Matemática esencial

nodo	w	b	u	$\hat{y}$	r	q	L
valor	1	0	2	2	-3	9	4.5
$\partial L / \partial(\cdot)$	-6	-3	-3	-3	-3	1/2	1

Interpretación

La fila de valores corresponde al cálculo hacia adelante. La fila de derivadas corresponde al efecto total de cada nodo sobre la pérdida final.

IDEA CLAVE

Una implementación es confiable cuando reproduce valores y gradientes esperados y puede explicar la ruta causal que los genera.

CUIDADO CON ESTA CONFUSIÓN

Que el código ejecute sin error no demuestra que el grafo o las derivadas sean correctos.

IV · DAG y autodiferenciación

Slide 13 - El orden topológico reúne contribuciones antes de propagar

PRIMERO ENTIÉNDELO ASÍ

En un grafo, un nodo puede depender de varios nodos anteriores y también puede alimentar varias rutas posteriores. Para hacer backward correctamente necesitamos procesar cada nodo solo cuando ya se han reunido todas las contribuciones que llegan desde sus descendientes.

QUÉ DEBES COMPRENDER

El **orden topológico** es un orden compatible con las dependencias: en el forward cada nodo aparece después de sus padres. Para backward se recorre ese orden al revés.

Matemática esencial

forward posible: $[w, x, u, b, \hat{y}, y, r, q, L]$, backward: orden inverso.

Interpretación

La topología decide *cuándo* puede procesarse un nodo. La regla de la cadena decide *cuánto* aporta cada ruta. La suma reúne los aportes cuando existen varias rutas.

Ejemplo breve

Si un nodo a influye en la salida por dos ramas, su sensibilidad total debe ser la suma de ambas antes de continuar propagando hacia los nodos anteriores.

IDEA CLAVE

El orden topológico no calcula derivadas por sí mismo; garantiza que las dependencias se respeten durante el recorrido.

CUIDADO CON ESTA CONFUSIÓN

Procesar un nodo antes de recibir todas sus contribuciones produce un gradiente incompleto.

Slide 14 - El lote cambia las formas, no la pérdida escalar

PRIMERO ENTIÉNDELO ASÍ

Pasar de una observación a un lote significa producir varias predicciones y varios residuos al mismo tiempo. Sin embargo, después de la reducción seguimos obteniendo una sola pérdida escalar.

QUÉ DEBES COMPRENDER

El eje del lote contiene observaciones. La pérdida cuadrática por lotes calcula un error por observación y luego los combina mediante suma y promedio.

Matemática esencial

$$\hat{y} = Xw + b\mathbf{1}_B, \quad r = \hat{y} - y, \quad L = \frac{1}{2B} \|r\|_2^2 \in \mathbb{R}.$$

$$X = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad y = \begin{bmatrix} 3 \\ 5 \\ 7 \end{bmatrix}, \quad w_0 = 1, \quad b_0 = 0 \Rightarrow r = \begin{bmatrix} -2 \\ -3 \\ -4 \end{bmatrix}, \quad L_0 = \frac{29}{6}.$$

Interpretación

B es el número de observaciones. X organiza una observación por fila. $\mathbf{1}_B$ replica el mismo sesgo en todas las observaciones. El residuo es un vector; la norma al cuadrado y la división por $2B$ lo reducen a un escalar.

IDEA CLAVE

El forward puede trabajar con vectores y matrices mientras el objeto final de entrenamiento siga siendo una pérdida escalar.

CUIDADO CON ESTA CONFUSIÓN

Pérdida escalar no significa residuo escalar. El residuo conserva el eje del lote hasta la reducción final.

Slide 15 - Los gradientes se acumulan en las hojas del grafo

PRIMERO ENTIÉNDELO ASÍ

En PyTorch, los parámetros creados como hojas con `requires_grad=True` reciben sus gradientes en el atributo `.grad`. Los tensores intermedios conservan la historia del cálculo mediante `grad_fn`, pero normalmente no guardan su gradiente en `.grad`.

QUÉ DEBES COMPRENDER

Una **hoja** es un tensor que no fue producido por otra operación rastreada y que actúa como punto de partida del grafo, por ejemplo un parámetro creado directamente.

Matemática esencial

$$\text{lote del ejemplo:} \quad \frac{\partial L}{\partial w} = -\frac{20}{3}, \quad \frac{\partial L}{\partial b} = -3.$$

Interpretación

Después de `loss.backward()`, esos valores se acumulan en `w.grad` y `b.grad`. Saber dónde se almacenan es parte de entender el comportamiento de autograd.

Código esencial

```
w = torch.tensor(1.0, requires_grad=True)
b = torch.tensor(0.0, requires_grad=True)
loss = ...
loss.backward()
print(w.grad, b.grad)
```

IDEA CLAVE

Autograd recorre el grafo; los gradientes que normalmente interesan para actualizar el modelo quedan en las hojas entrenables.

CUIDADO CON ESTA CONFUSIÓN

No esperes que cada tensor intermedio tenga un valor visible en `.grad` de forma predeterminada.

Slide 16 - Acumular y reiniciar son decisiones distintas

PRIMERO ENTIÉNDELO ASÍ

PyTorch suma las nuevas contribuciones sobre el contenido existente de `.grad`. Por eso, ejecutar `backward()` varias veces sin reiniciar puede hacer que los gradientes se dupliquen o acumulen.

QUÉ DEBES COMPRENDER

Acumular significa añadir el nuevo gradiente al valor almacenado. Reiniciar significa preparar los campos `.grad` para que el siguiente `backward` represente solo la nueva contribución.

Matemática esencial

$$g \xrightarrow{\text{backward}} g \text{ y otra vez sin reinicio } \longrightarrow 2g.$$

Interpretación

`optimizer.zero_grad(set_to_none=True)` puede dejar `.grad=None`. Eso significa que todavía no se ha registrado una contribución nueva. Un tensor de ceros, en cambio, es un gradiente numérico cuyo valor es cero.

Código esencial

```
optimizer.zero_grad(set_to_none=True)
loss.backward()
optimizer.step()
```

IDEA CLAVE

El reinicio de gradientes es una decisión explícita del ciclo de entrenamiento; `backward()` no lo hace automáticamente.

CUIDADO CON ESTA CONFUSIÓN

Acumular entre llamadas distintas a `backward` es diferente de sumar rutas dentro de un único grafo.

V · PyTorch, lotes y actualización

Slide 17 - El término lineal revela el gradiente del lote

PRIMERO ENTÉNDELO ASÍ

Para obtener el gradiente por lotes podemos imaginar un cambio pequeño Δw en los pesos y observar cómo ese cambio modifica el residuo y la pérdida.

QUÉ DEBES COMPRENDER

El término que es lineal en Δw indica el cambio de primer orden. Su coeficiente es precisamente el gradiente respecto de w .

Matemática esencial

$$r(w + \Delta w, b) = r + X \Delta w.$$

$$L(w + \Delta w, b) = L(w, b) + \frac{1}{B} r^T X \Delta w + \frac{1}{2B} \|X \Delta w\|_2^2.$$

$$\boxed{\nabla_w L = \frac{1}{B} X^T r}.$$

Interpretación

r es el residuo en los pesos actuales. $X \Delta w$ describe cómo cambian todas las predicciones al mover los pesos. El término cuadrático es de orden superior; el término lineal contiene la pendiente que interesa para cambios pequeños.

IDEA CLAVE

El gradiente aparece como el coeficiente que multiplica al pequeño cambio Δw en la expansión de la pérdida.

CUIDADO CON ESTA CONFUSIÓN

No confundas $X^T r$ con una operación arbitraria: sus dimensiones hacen que el resultado tenga la misma forma que w .

Slide 18 - Cada gradiente hereda la forma de su parámetro

PRIMERO ENTIÉNDELO ASÍ

Un parámetro vectorial necesita un gradiente vectorial de la misma forma; un sesgo escalar necesita un gradiente escalar. La forma del gradiente indica qué cantidad puede actualizar directamente.

QUÉ DEBES COMPRENDER

La regla de formas funciona como una auditoría: si el gradiente calculado no tiene la misma forma que su parámetro, probablemente existe un error de orientación o broadcasting.

Matemática esencial

$$\nabla_w L = \frac{1}{B} X^T r \in \mathbb{R}^d, \quad \frac{\partial L}{\partial b} = \frac{1}{B} \mathbf{1}_B^T r \in \mathbb{R}.$$

Interpretación

Si $X \in \mathbb{R}^{B \times d}$ y $r \in \mathbb{R}^B$, entonces $X^T r$ pertenece a $\mathbb{R}^d$, exactamente como w . El término $\mathbf{1}_B^T r$ suma las contribuciones del sesgo y produce un escalar.

IDEA CLAVE

Gradiente y parámetro deben ser compatibles en forma porque la actualización combina ambos elemento a elemento.

CUIDADO CON ESTA CONFUSIÓN

Que PyTorch acepte una operación por broadcasting no garantiza que la función objetivo represente lo que se pretendía.

V · PyTorch, lotes y actualización

Slide 19 - Un paso explícito verifica la derivación antes de delegar

PRIMERO ENTIÉNDELO ASÍ

Una vez calculados los gradientes podemos realizar manualmente un paso de descenso y comprobar que los parámetros y las predicciones cambian en el sentido esperado.

QUÉ DEBES COMPRENDER

La tasa de aprendizaje η controla qué fracción del gradiente se convierte en movimiento efectivo del parámetro.

Matemática esencial

$$w_1 = w_0 - \eta \nabla_w L = 1 - 0.1 \left(-\frac{20}{3} \right) = \frac{5}{3}, \quad b_1 = b_0 - 0.1(-3) = 0.3.$$

$$\hat{y}_1 - \hat{y}_0 = X \left(\frac{2}{3} \right) + 0.3 \mathbf{1}_3 > 0.$$

Interpretación

Aquí $X = (1, 2, 3)^T$. El cambio $2/3$ en w afecta a cada observación multiplicándose por su x_i . El cambio 0.3 en b se suma por igual a las tres predicciones mediante $0.3\mathbf{1}_3$.

Código esencial

```
with torch.no_grad():
    w -= eta * w.grad
    b -= eta * b.grad
```

IDEA CLAVE

El gradiente proporciona la pendiente; η determina cuánto se aplica de esa corrección.

CUIDADO CON ESTA CONFUSIÓN

`torch.no_grad()` evita registrar la actualización como una nueva operación del grafo; no borra los gradientes ya almacenados.

VI · Estabilidad de gradient descent

Slide 20 - El descenso en una cuadrática produce una recurrencia del error

PRIMERO ENTIÉNDELO ASÍ

Para estudiar qué ocurre al repetir Gradient Descent (GD) muchas veces usamos una pérdida cuadrática de una sola variable. La ventaja es que el comportamiento puede describirse exactamente mediante una regla muy simple para el error.

QUÉ DEBES COMPRENDER

Aquí **GD** significa *Gradient Descent*, descenso por gradiente. θ^* es el valor del parámetro que minimiza la pérdida.

Matemática esencial

$$L(\theta) = \frac{a}{2}(\theta - \theta^*)^2, \quad L'(\theta) = a(\theta - \theta^*), \quad a > 0.$$

$$\theta_{t+1} = \theta_t - \eta a(\theta_t - \theta^*).$$

$$e_t = \theta_t - \theta^* \quad \Rightarrow \quad \boxed{e_{t+1} = (1 - \eta a)e_t}.$$

Interpretación

e_t no es la pérdida: es la distancia con signo entre el parámetro actual y el óptimo. En cada iteración esa distancia se multiplica por el mismo factor $\rho = 1 - \eta a$.

Ejemplo breve

Si $a = 4$ y $\eta = 0.1$, entonces $\rho = 0.6$. Un error de 5 pasa a 3, después a 1.8: en cada paso queda el 60 % del error anterior.

IDEA CLAVE

La estabilidad de GD sobre esta cuadrática se reduce a estudiar el número $\rho = 1 - \eta a$.

CUIDADO CON ESTA CONFUSIÓN

El signo de e_t indica a qué lado del óptimo estamos; su magnitud indica la distancia al óptimo.

VI · Estabilidad de gradient descent

Slide 21 - Cinco regímenes de la tasa de aprendizaje

PRIMERO ENTIÉNDELO ASÍ

La recurrencia $e_{t+1} = \rho e_t$ permite saber si el error se reduce, oscila o crece. Todo depende del signo y de la magnitud de $\rho = 1 - \eta a$.

QUÉ DEBES COMPRENDER

Oscilar significa que el signo del error cambia y el parámetro salta de un lado al otro del óptimo. Converger significa que $|e_t|$ tiende a cero.

Matemática esencial

$\rho = 1 - \eta a.$	
rango de ρ	comportamiento
$\rho < -1$	diverge alternando
$\rho = -1$	oscila sin reducirse
$-1 < \rho < 0$	oscila y converge
$\rho = 0$	llega al óptimo en un paso
$0 < \rho < 1$	converge sin cambiar de lado

Interpretación

La condición de convergencia es $|\rho| < 1$. Sustituyendo $\rho = 1 - \eta a$ se obtiene $0 < \eta < 2/a$.

IDEA CLAVE

La tasa de aprendizaje debe ser compatible con la escala a de la cuadrática; una tasa demasiado grande puede hacer divergir el proceso.

CUIDADO CON ESTA CONFUSIÓN

a representa cuán rápidamente cambia la pendiente al alejarnos del mínimo: un a mayor hace que la parábola sea más cerrada y que el gradiente crezca más deprisa.

VI · Estabilidad de gradient descent

Slide 22 - La mayor curvatura fija el límite estable

PRIMERO ENTIÉNDELO ASÍ

En varias dimensiones, una cuadrática puede ser más cerrada en unas direcciones que en otras. Cada dirección principal tiene su propia escala de curvatura, dada por un autovalor de la matriz Hessiana.

QUÉ DEBES COMPRENDER

Aquí **curvatura** se usa de forma concreta: mide qué tan rápido cambia la pendiente al movernos en una dirección. Una dirección con mayor curvatura exige pasos más pequeños para mantener estabilidad.

Matemática esencial

$$L(\theta) = \frac{1}{2}(\theta - \theta^*)^T H(\theta - \theta^*).$$

$$e_{i,t+1} = (1 - \eta\lambda_i)e_{i,t}.$$

$$0 < \eta < \frac{2}{\lambda_{\max}(H)}.$$

Interpretación

H es la Hessiana de la cuadrática y λ_i sus autovalores. Cada componente del error, expresada en una dirección propia de H , se comporta como el caso unidimensional. Para que todas sean estables, la tasa debe respetar la dirección con el mayor λ_i .

IDEA CLAVE

La dirección de mayor curvatura impone el límite común de la tasa de aprendizaje en una cuadrática positiva definida.

CUIDADO CON ESTA CONFUSIÓN

Este resultado es exacto para cuadráticas. En funciones generales se usa como intuición local, no como una garantía global automática.

VI · Estabilidad de gradient descent

Slide 23 - Una tabla de tasas convierte teoría en predicciones falsables

PRIMERO ENTIÉNDELO ASÍ

La fórmula $\rho = 1 - \eta a$ permite predecir el comportamiento antes de ejecutar código. Con $a = 4$, basta sustituir diferentes tasas de aprendizaje.

QUÉ DEBES COMPRENDER

Una predicción es falsable cuando el código puede contradecirla de manera clara. Aquí se predice si el error debe disminuir, cambiar de signo o crecer.

Matemática esencial

$a = 4, \quad \rho = 1 - 4\eta.$		
η	ρ	predicción
0.10	0.60	converge sin oscilar
0.25	0	un paso
0.40	-0.60	oscila y converge
0.50	-1	oscila sin reducirse
0.60	-1.40	diverge

Interpretación

La misma regla algebraica explica todos los casos. El signo de ρ determina si se cambia de lado; $|\rho|$ determina si la distancia al óptimo disminuye o aumenta.

IDEA CLAVE

La teoría permite anticipar patrones observables y convertir la ejecución en una comprobación, no en una exploración a ciegas.

CUIDADO CON ESTA CONFUSIÓN

No evalúes solo si la pérdida baja al principio; algunas tasas pueden alternar o divergir después de varios pasos.

VII · Mini-lotes y optimizadores

Slide 24 - El mini-lote es un estimador del gradiente completo

PRIMERO ENTIÉNDELO ASÍ

Calcular el gradiente con todos los ejemplos puede ser costoso. Un mini-lote usa solo una parte de los datos para producir una aproximación del gradiente completo en la iteración actual.

QUÉ DEBES COMPRENDER

Un **estimador** es una cantidad calculada a partir de una muestra para aproximar otra cantidad. Decir que es **insesgado** significa que, al promediar sobre muchas selecciones aleatorias bajo las hipótesis de muestreo, su valor esperado coincide con el gradiente completo.

Matemática esencial

$$L(\theta) = \frac{1}{B} \sum_{i=1}^B \ell_i(\theta), \quad \nabla L(\theta) = \frac{1}{B} \sum_{i=1}^B \nabla \ell_i(\theta).$$

$$g_t = \frac{1}{m} \sum_{i \in S_t} \nabla \ell_i(\theta_t), \quad \mathbb{E}[g_t \mid \theta_t] = \nabla L(\theta_t).$$

Interpretación

S_t es el conjunto de índices del mini-lote y m su tamaño. Un mini-lote concreto puede producir un gradiente bastante distinto del completo; la propiedad de insesgo habla del promedio sobre selecciones posibles.

IDEA CLAVE

Mini-lote no significa copiar una parte exacta del gradiente completo: produce una estimación que varía con la muestra.

CUIDADO CON ESTA CONFUSIÓN

La igualdad en esperanza depende de cómo se seleccionan las observaciones; no afirma que cualquier esquema de muestreo sea automáticamente insesgado.

VII · Mini-lotes y optimizadores

Slide 25 - Tamaño de mini-lote: costo frente a variabilidad

PRIMERO ENTIÉNDELO ASÍ

El tamaño del mini-lote controla un compromiso práctico. Lotes pequeños requieren menos trabajo por paso pero el gradiente cambia más entre selecciones; lotes grandes cuestan más por paso y suelen aproximarse mejor al gradiente completo.

QUÉ DEBES COMPRENDER

Variabilidad significa que dos mini-lotes diferentes, evaluados en los mismos parámetros, pueden producir gradientes distintos.

Matemática esencial

$$x_i r_i \in \{-2, -6, -12\}, \quad g_{\text{full}} = \frac{-2 - 6 - 12}{3} = -\frac{20}{3}.$$

$$m = 2 : \quad \{-2, -6\} \rightarrow -4, \quad \{-6, -12\} \rightarrow -9, \quad \{-2, -12\} \rightarrow -7.$$

Interpretación

Las tres estimaciones de tamaño dos son diferentes entre sí y también diferentes del gradiente completo. Ese cambio entre muestras es parte normal del uso de mini-lotes.

IDEA CLAVE

Reducir el lote disminuye el costo inmediato de cada paso, pero aumenta la variabilidad del estimador del gradiente.

CUIDADO CON ESTA CONFUSIÓN

No existe un tamaño universalmente mejor; el compromiso depende del problema, hardware y dinámica de optimización.

VII · Mini-lotes y optimizadores

Slide 26 - Momentum añade memoria; no cambia el gradiente**PRIMERO ENTIÉNDELO ASÍ**

Gradient Descent simple usa solo el gradiente actual. Momentum conserva una cantidad entre iteraciones y combina esa memoria con el gradiente nuevo antes de decidir el paso.

QUÉ DEBES COMPRENDER

Aquí **memoria** o **estado** significa literalmente un valor numérico que el optimizador guarda de una iteración a la siguiente. No es una propiedad del gradiente calculado por autograd.

Matemática esencial

$$v_t = \mu v_{t-1} + g_t, \quad \theta_{t+1} = \theta_t - \eta v_t, \quad 0 \leq \mu < 1.$$

Interpretación

g_t es el gradiente actual. v_t mezcla el historial previo v_{t-1} con el gradiente nuevo. μ controla cuánto pesa la memoria y η controla el tamaño global del paso.

IDEA CLAVE

Momentum no modifica cómo se calcula g_t ; modifica cómo ese gradiente se transforma en una actualización.

CUIDADO CON ESTA CONFUSIÓN

No identifiques v_t con el gradiente actual: puede tener magnitud e incluso signo diferentes debido a la memoria acumulada.

VII · Mini-lotes y optimizadores

Slide 27 - La memoria puede agrandar el paso aunque el gradiente se reduzca**PRIMERO ENTIÉNDELO ASÍ**

El efecto de momentum se entiende mejor observando una secuencia. Un gradiente actual más pequeño no implica necesariamente un paso más pequeño, porque el estado previo también contribuye.

QUÉ DEBES COMPRENDER

El paso aplicado depende de v_t , no directamente de g_t . Por eso la historia reciente puede reforzar o contrarrestar al gradiente actual.

Matemática esencial

$$v_0 = 0, \quad \mu = 0.9, \quad \eta = 0.1, \quad (g_1, g_2, g_3) = (-3, -1, 4). \\ v_1 = -3, \Delta\theta_1 = +0.30; \quad v_2 = -3.7, \Delta\theta_2 = +0.37; \quad v_3 = 0.67, \Delta\theta_3 = -0.067.$$

Interpretación

En el segundo paso el gradiente baja en magnitud de 3 a 1, pero la memoria hace que $|v_2|$ aumente a 3.7. En el tercero, el gradiente positivo termina invirtiendo el signo del estado.

IDEA CLAVE

Con momentum, el tamaño y signo del paso dependen de una combinación del presente y del pasado reciente.

CUIDADO CON ESTA CONFUSIÓN

No deduzcas el movimiento del parámetro mirando solo el gradiente de una iteración.

VII · Mini-lotes y optimizadores

Slide 28 - El momentum_buffer hace visible la memoria del optimizador

PRIMERO ENTIÉNDELO ASÍ

PyTorch almacena el estado de momentum dentro del optimizador. Ese estado puede inspeccionarse y compararse con la recurrencia matemática para verificar que la implementación sigue la regla esperada.

QUÉ DEBES COMPRENDER

El momentum_buffer es la representación concreta de la memoria que persiste entre llamadas a `optimizer.step()`.

Matemática esencial

$$v_t = \mu v_{t-1} + g_t.$$

Interpretación

Si se inyectan gradientes controlados, el buffer debe evolucionar de acuerdo con esa recurrencia. Observar el buffer permite distinguir el gradiente instantáneo del estado acumulado.

Código esencial

```
opt = torch.optim.SGD([p], lr=0.1, momentum=0.9)
# después de asignar p.grad y llamar opt.step():
buf = opt.state[p]['momentum_buffer']
```

IDEA CLAVE

El estado del optimizador es observable y forma parte de la explicación de por qué el paso actual tiene cierta magnitud.

CUIDADO CON ESTA CONFUSIÓN

El buffer no es una copia del gradiente actual; incorpora información de pasos anteriores.

VII · Mini-lotes y optimizadores

Slide 29 - Adam mantiene dos memorias y corrige su inicio

PRIMERO ENTÉNDELO ASÍ

Adam mantiene dos estados por parámetro: uno resume el gradiente y otro resume el cuadrado del gradiente. Después corrige el sesgo inicial producido por comenzar ambos estados en cero.

QUÉ DEBES COMPRENDER

El primer estado ayuda a seguir una dirección suavizada; el segundo mide una escala reciente de magnitudes por coordenada. Esto permite adaptar el paso de forma diferente en cada componente.

Matemática esencial

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

Interpretación

m_t y v_t son estados del optimizador, no gradientes nuevos calculados por autograd. Las correcciones con $1 - \beta_1^t$ y $1 - \beta_2^t$ compensan que los promedios móviles comenzaron en cero.

IDEA CLAVE

Adam transforma el mismo gradiente usando dos memorias internas y una normalización por coordenada.

CUIDADO CON ESTA CONFUSIÓN

El símbolo v_t en Adam representa el segundo momento exponencial; no es necesariamente el mismo objeto conceptual que la variable de momentum usada en la slide 26.

VII · Mini-lotes y optimizadores

Slide 30 - Autograd calcula el gradiente; Adam decide el paso

PRIMERO ENTIÉNDELO ASÍ

La secuencia de una iteración separa claramente dos responsabilidades. Primero autograd calcula g durante `backward()`. Después Adam actualiza sus estados y decide el cambio del parámetro durante `step()`.

QUÉ DEBES COMPRENDER

El campo `parameter.grad` conserva el gradiente calculado. El estado de Adam vive en el optimizador y cambia cuando se ejecuta el paso.

Matemática esencial

$$p_0 = 1, \quad y = 3, \quad L = \frac{1}{2}(p - 3)^2 = 2, \quad g = \frac{dL}{dp} = -2.$$

con Adam y $\eta = 0.01$: $p_1 \approx 1.01$.

Interpretación

El primer paso de Adam normaliza el gradiente con sus estimaciones corregidas; por eso el cambio puede ser cercano a 0.01 aunque el gradiente sea -2 .

Código esencial

```
optimizer.zero_grad()
loss.backward() # calcula p.grad
optimizer.step() # actualiza estado y p
```

IDEA CLAVE

`backward()` produce el gradiente; `step()` produce la actualización del parámetro.

CUIDADO CON ESTA CONFUSIÓN

Después de `step()`, `p.grad` no se borra automáticamente. Sigue conteniendo el gradiente anterior hasta que se reinicie.

VIII · train/eval y registro del grafo

Slide 31 - Modo del módulo y registro del grafo son interruptores distintos**PRIMERO ENTIÉNDELO ASÍ**

PyTorch tiene dos decisiones independientes. `model.train()` y `model.eval()` controlan el comportamiento de ciertos módulos. El modo de gradientes controla si las operaciones se registran para autograd.

QUÉ DEBES COMPRENDER

Estos interruptores son ortogonales: activar evaluación no implica desactivar el grafo, y usar `no_grad()` no cambia por sí mismo el modo del módulo.

Matemática esencial

modo del módulo	gradientes habilitados	resultado típico
train	sí	training=True, grad_fn presente
train	no	training=True, grad_fn=None
eval	sí	training=False, grad_fn presente
eval	no	training=False, grad_fn=None

Interpretación

El primer eje cambia reglas internas de módulos como Dropout o BatchNorm. El segundo decide si PyTorch construye la historia necesaria para derivar.

IDEA CLAVE

`eval()` no equivale a `no_grad()`. Son controles diferentes y pueden combinarse de cuatro maneras.

CUIDADO CON ESTA CONFUSIÓN

La ausencia o presencia de historial de autograd no debe inferirse únicamente a partir de `model.training`.

Slide 32 - Dropout consulta training; una capa Linear no lo hace

PRIMERO ENTIÉNDELO ASÍ

No todos los módulos cambian su fórmula entre entrenamiento y evaluación. Dropout sí consulta el modo `training`; una capa Linear aplica la misma transformación afín en ambos modos.

QUÉ DEBES COMPRENDER

Con Dropout y $p = 0.5$, durante entrenamiento algunos elementos se anulan y los restantes se reescalan. En evaluación Dropout se convierte en la identidad.

Matemática esencial

Linear: $y = xW^T + b$ en train y eval.

Interpretación

Para una entrada de unos y Dropout con $p = 0.5$, durante entrenamiento cada componente puede convertirse en 0 o 2. En evaluación la salida de Dropout coincide con la entrada.

IDEA CLAVE

El modo del módulo afecta solo a las capas cuya definición depende de `training`; no cambia automáticamente todas las operaciones del modelo.

CUIDADO CON ESTA CONFUSIÓN

Un tensor puede seguir teniendo `requires_grad=True` en modo evaluación.

Slide 33 - El reescalamiento de Dropout preserva la expectativa

PRIMERO ENTIÉNDELO ASÍ

Durante entrenamiento, Dropout no solo elimina elementos al azar: también reescala los que sobreviven. El objetivo es que el valor esperado de cada componente se mantenga igual al de la entrada.

QUÉ DEBES COMPRENDER

Valor esperado significa el promedio teórico que obtendríamos al repetir muchas veces el muestreo de la máscara aleatoria.

Matemática esencial

$$\tilde{x} = \frac{m}{1-p}x, \quad m \sim \text{Bernoulli}(1-p).$$
$$\mathbb{E}[\tilde{x}] = \frac{\mathbb{E}[m]}{1-p}x = x.$$

Interpretación

Con $x = 1$ y $p = 0.5$, m vale 0 o 1 con probabilidades 0.5. Por tanto $\tilde{x}$ vale 0 o 2, y su promedio teórico es 1.

IDEA CLAVE

El reescalamiento invertido conserva la expectativa de la activación individual entre entrenamiento y evaluación.

CUIDADO CON ESTA CONFUSIÓN

Conservar la expectativa no significa conservar cada muestra, la varianza ni la salida completa de una red no lineal.

Slide 34 - El modo evaluación no desactiva autograd

PRIMERO ENTIÉNDELO ASÍ

Poner el modelo en `eval()` modifica el comportamiento de módulos sensibles al modo, pero autograd puede seguir registrando operaciones y calculando derivadas.

QUÉ DEBES COMPRENDER

La evidencia correcta se observa en propiedades como `requires_grad`, `grad_fn` y en la posibilidad de ejecutar `backward` hasta una entrada o parámetro.

Matemática esencial

$$\text{model.eval()} + \text{gradientes habilitados} \Rightarrow \text{autograd sigue activo.}$$

Interpretación

Este caso es útil cuando se quiere un comportamiento determinista de evaluación pero todavía se necesitan derivadas, por ejemplo para análisis de sensibilidad o métodos de atribución.

IDEA CLAVE

`eval()` controla el modo del módulo; no es un interruptor global de derivación.

CUIDADO CON ESTA CONFUSIÓN

No concluyas que no habrá gradientes solo porque `model.training` sea `False`.

VIII · train/eval y registro del grafo

Slide 35 - no_grad() corta la historia de las operaciones internas

PRIMERO ENTIÉNDELO ASÍ

Dentro de `torch.no_grad()`, las operaciones nuevas no se registran en el grafo. Aunque una entrada requiera gradientes, una operación realizada dentro del bloque puede producir un tensor sin conexión histórica hacia esa entrada.

QUÉ DEBES COMPRENDER

Cortar la historia significa que autograd ya no conserva el camino de operaciones necesario para regresar desde resultados posteriores hasta el tensor original.

Matemática esencial

$$x \xrightarrow[\text{no_grad}]{z=3x} z \xrightarrow[\text{grad habilitado}]{q=4z} q.$$

Interpretación

Si z se creó dentro de `no_grad()`, entonces normalmente `z.requires_grad=False` y `z.grad_fn=None`. Calcular después $q = 4z$ fuera del bloque no reconstruye la historia perdida hasta x .

Código esencial

```
x = torch.tensor(2., requires_grad=True)
with torch.no_grad():
    z = 3 * x
    q = 4 * z
```

IDEA CLAVE

`no_grad()` afecta a las operaciones realizadas dentro del contexto; no cambia retroactivamente la propiedad original de x .

CUIDADO CON ESTA CONFUSIÓN

Salir del bloque vuelve a habilitar el registro para operaciones futuras, pero no reconecta operaciones que ya se ejecutaron sin historial.

VIII · train/eval y registro del grafo

Slide 36 - Las cuatro combinaciones se verifican con la misma capa**PRIMERO ENTIÉNDELO ASÍ**

Usar la misma capa en las cuatro combinaciones permite separar con evidencia qué controla el modo del módulo y qué controla el registro del grafo.

QUÉ DEBES COMPRENDER

Una capa Linear es útil para esta comprobación porque su fórmula no cambia entre train y eval; así el foco queda en `model.training`, `requires_grad` y `grad_fn`.

Matemática esencial

$$\{\text{train, eval}\} \times \{\text{grad, no-grad}\}.$$

Interpretación

Con gradientes habilitados, una salida que depende de entradas o parámetros entrenables conserva historial. Con `no_grad()`, esa historia no se crea. Cambiar train por eval solo modifica el atributo de modo y las capas que lo consultan.

IDEA CLAVE

Los cuatro casos confirman que modo del módulo y modo de gradientes son dimensiones independientes.

CUIDADO CON ESTA CONFUSIÓN

Si la capa elegida cambia su fórmula entre train y eval, como Dropout, se mezclan dos efectos y la comparación es menos limpia.

VIII · train/eval y registro del grafo

Slide 37 - La intención determina modo y contexto**PRIMERO ENTIÉNDELO ASÍ**

La combinación adecuada depende de lo que se pretende hacer. No existe una sola configuración correcta para todos los usos del modelo.

QUÉ DEBES COMPRENDER

La decisión puede organizarse por intención: entrenar, actualizar parámetros manualmente, ejecutar inferencia o calcular derivadas durante evaluación.

Matemática esencial

intención	configuración
entrenamiento	<code>model.train()</code> + gradientes habilitados
actualización manual	<code>torch.no_grad()</code> alrededor del cambio
inferencia	<code>model.eval()</code> + <code>torch.no_grad()</code>
derivadas en evaluación	<code>model.eval()</code> + gradientes habilitados

Interpretación

Cada combinación expresa una necesidad distinta: comportamiento de capas, disponibilidad de derivadas y costo de almacenar el grafo.

IDEA CLAVE

Primero identifica la intención; después elige el modo del módulo y el contexto de gradientes apropiados.

CUIDADO CON ESTA CONFUSIÓN

Evita modificar parámetros mediante `.data`; un bloque `no_grad()` expresa la intención de forma más segura y explícita.

IX · Ciclo, evaluación y diagnóstico

Slide 38 - Un ciclo mínimo separa responsabilidades observables

PRIMERO ENTIÉNDELO ASÍ

Un ciclo de entrenamiento correcto puede leerse como una cadena causal: limpiar gradientes, producir predicciones, calcular una pérdida escalar, derivar, actualizar parámetros y registrar evidencia del cambio.

QUÉ DEBES COMPRENDER

Separar etapas permite localizar fallos. Si la pérdida existe pero los parámetros no cambian, se puede comprobar si el problema está en los gradientes, en el optimizador o en la actualización.

Matemática esencial
$$\text{zero_grad} \rightarrow \text{forward} \rightarrow L \rightarrow \text{backward} \rightarrow \nabla L \rightarrow \text{step} \rightarrow \text{nuevos parámetros.}$$
Interpretación

Métricas como loss, norma del gradiente y norma del cambio de parámetros aportan evidencia diferente: una mide el objetivo, otra la señal de derivación y otra el movimiento efectivo.

Código esencial

```
optimizer.zero_grad()
y_hat = model(x)
loss = criterion(y_hat, y)
loss.backward()
optimizer.step()
```

IDEA CLAVE

Un ciclo entendible expone qué responsabilidad corresponde a cada operación, no solo que el notebook termina sin error.

CUIDADO CON ESTA CONFUSIÓN

La evaluación usa una rama diferente: normalmente `eval()` y `no_grad()`, sin `backward()` ni `step()`.

IX · Ciclo, evaluación y diagnóstico

Slide 39 - La evaluación no añade ni elimina gradientes almacenados**PRIMERO ENTIÉNDELO ASÍ**

Evaluar con `model.eval()` y `torch.no_grad()` evita construir nueva historia de autograd y evita añadir nuevas contribuciones. Sin embargo, los valores que ya estaban guardados en `parameter.grad` permanecen allí.

QUÉ DEBES COMPRENDER

No registrar operaciones nuevas y borrar gradientes son acciones diferentes. La primera la controla `no_grad()`; la segunda requiere reiniciar explícitamente los campos de gradiente.

Matemática esencial

$$g_{\text{antes}} \xrightarrow{\text{eval} + \text{no-grad}} g_{\text{después}} = g_{\text{antes}}$$

Interpretación

Si se compara el contenido de `.grad` antes y después de una evaluación sin backward, debería conservarse el mismo valor.

IDEA CLAVE

La evaluación no limpia automáticamente los gradientes acumulados del entrenamiento anterior.

CUIDADO CON ESTA CONFUSIÓN

Ver un valor en `.grad` durante evaluación no demuestra que la evaluación haya calculado ese gradiente. Puede ser un valor previo.

IX · Ciclo, evaluación y diagnóstico

Slide 40 - Una curva descendente no basta para validar el entrenamiento

PRIMERO ENTIÉNDELO ASÍ

Que la pérdida disminuya es una señal útil, pero no demuestra por sí sola que el protocolo sea correcto. Una función objetivo equivocada, broadcasting accidental o fuga de información también pueden producir una curva aparentemente favorable.

QUÉ DEBES COMPRENDER

Validar el entrenamiento requiere seguir la cadena causal completa y comprobar que cada etapa representa lo que se pretende calcular.

Matemática esencial

objetivo correcto $\Rightarrow$ gradientes correctos $\Rightarrow$ actualización observable $\Rightarrow$ evaluación válida.

Interpretación

Una auditoría mínima incluye la fórmula y reducción de la pérdida, las formas de los tensores, propiedades de las hojas, posición de `zero_grad()`, estado del optimizador, movimiento de parámetros y configuración de evaluación.

IDEA CLAVE

La curva de pérdida es una evidencia entre varias; no reemplaza la verificación de formas, gradientes, estado y separación entre entrenamiento y evaluación.

CUIDADO CON ESTA CONFUSIÓN

Una pérdida que baja puede corresponder a la función equivocada. La ejecución exitosa no valida automáticamente la semántica del cálculo.

IX · Ciclo, evaluación y diagnóstico

Slide 41 - El broadcasting silencioso puede cambiar la función objetivo**PRIMERO ENTIÉNDELO ASÍ**

PyTorch puede combinar tensores de formas diferentes mediante broadcasting. A veces esto es exactamente lo deseado; otras veces cambia el emparejamiento entre predicción y objetivo sin producir un error de ejecución.

QUÉ DEBES COMPRENDER

Broadcasting significa expandir dimensiones compatibles de forma implícita para poder realizar una operación. El riesgo aparece cuando esa expansión no coincide con la intención matemática.

Matemática esencial

$$\hat{y} : (B,), \quad y : (B, 1) \quad \Rightarrow \quad \hat{y} - y : (B, B).$$

Interpretación

En lugar de obtener B residuos correspondientes a pares $(\hat{y}_i, y_i)$, se obtiene una matriz con diferencias entre muchas parejas cruzadas. La pérdida resultante optimiza otro objetivo.

Código esencial

```
print(pred.shape, target.shape)
# correcto: ambos (B,) o ambos (B, 1)
```

IDEA CLAVE

Antes de calcular la pérdida, las formas de predicción y objetivo deben representar el mismo emparejamiento de observaciones.

CUIDADO CON ESTA CONFUSIÓN

Que una operación sea válida para PyTorch no implica que sea válida para el problema matemático que se quería modelar.

IX · Ciclo, evaluación y diagnóstico

Slide 42 - Tres síntomas exigen tres rutas diagnósticas**PRIMERO ENTIÉNDELO ASÍ**

Distintos síntomas apuntan a fallos diferentes. Diagnosticar significa comprobar primero la causa más directamente relacionada con la evidencia observada, en lugar de cambiar hiperparámetros al azar.

QUÉ DEBES COMPRENDER

Una ruta diagnóstica conecta síntoma, causa plausible y primera comprobación observable.

Matemática esencial

síntoma	primera comprobación
<code>parameter.grad is None</code>	<code>requires_grad, is_leaf, grad_fn</code>
gradiente aproximadamente doble	posición de <code>zero_grad()</code>
<code>change_norm = 0</code>	g , η , parámetros registrados e identidad

Interpretación

`grad is None` sugiere falta de conexión o de seguimiento. Un gradiente duplicado sugiere acumulación entre backward. Un cambio nulo puede deberse a gradiente cero, tasa cero o a que el optimizador no está actualizando los parámetros que se están inspeccionando.

IDEA CLAVE

El diagnóstico mejora cuando cada hipótesis se vincula con una observación concreta que puede confirmarla o descartarla.

CUIDADO CON ESTA CONFUSIÓN

Reducir automáticamente el learning rate no corrige problemas de grafo, acumulación, registro de parámetros o formas.

IX · Ciclo, evaluación y diagnóstico

Slide 43 - El Hackathon exige evidencia causal, no solo un notebook que corre**PRIMERO ENTIÉNDELO ASÍ**

El cierre del módulo integra formulación matemática, predicción, ejecución y diagnóstico. El objetivo es poder explicar por qué el resultado observado es coherente con el grafo, los gradientes, el optimizador y el contexto de evaluación.

QUÉ DEBES COMPRENDER

La evidencia causal conecta una decisión con un efecto observable: una forma determina una operación, una operación determina un gradiente, el gradiente y el estado determinan un paso, y el paso modifica los parámetros.

Matemática esencial

formular → predecir → ejecutar → medir → diagnosticar → justificar.

Interpretación

Una solución sólida registra como mínimo la pérdida, la norma de gradiente y la magnitud del cambio de parámetros, además de las formas y configuraciones relevantes cuando existe un problema.

IDEA CLAVE

Un notebook correcto no solo produce una salida: permite reconstruir la cadena de causas que llevó a esa salida.

CUIDADO CON ESTA CONFUSIÓN

Una corrección debe explicar qué estaba mal, qué evidencia lo muestra, por qué el cambio lo corrige y qué límites sigue teniendo la solución.

Glosario mínimo del módulo

Gradiente	Vector que reúne la derivada de una pérdida escalar respecto de cada parámetro.
Cambio local	Cambio pequeño alrededor de los valores actuales de los parámetros.
Sensibilidad	Cuánto cambia la pérdida final si un nodo intermedio cambia ligeramente.
Backpropagation	Aplicación organizada de la regla de la cadena en sentido inverso sobre el grafo.
DAG	Grafo dirigido acíclico que representa dependencias entre operaciones.
Hoja	Tensor de origen del grafo, normalmente un parámetro creado directamente.
Estimador	Cantidad calculada con una muestra para aproximar otra cantidad, como el gradiente completo.
Estado del optimizador	Valores que un optimizador conserva entre iteraciones, por ejemplo momentum o los momentos de Adam.
Curvatura	En este módulo, rapidez con la que cambia la pendiente al movernos en una dirección.
Broadcasting	Expansión implícita de dimensiones compatibles para realizar operaciones entre tensores de formas diferentes.

IDEA GLOBAL

El módulo puede leerse como una sola cadena: una pérdida escalar define qué queremos reducir; el gradiente describe cómo cambia alrededor de los parámetros actuales; backpropagation calcula ese gradiente a través del grafo; el optimizador transforma la información en un paso; y la evaluación y el diagnóstico comprueban que todo el proceso representa la función objetivo correcta.